TEORÍA DE COMPILADORES · 1ACC0218 · SECCIÓN 4889 · HITO 2

Kipu

Un lenguaje de dominio específico para reglas tributarias y montos monetarios



Gramática actualizada, semántico completo, arquitectura, plan de validación y avance con LLVM

Bussalleu Salcedo, Fabrizio (U202315655) · Sancho Navarro, Jose Luis (U202016523)
Cabrera Díaz, Marco Antonio Luciano (U202318540)
Docente: Jose Luis Soncco Alvarez · UPC, Lima, noviembre de 2026

Repositorio: github.com/FabrizioBussalleu/kipu

PROBLEMÁTICA

Las reglas tributarias viven en Excel o en código genérico

IGV 18 %, detracciones del SPOT, retenciones, percepciones y tipo de cambio: todo se programa a mano en cada sistema de facturación.

01

Punto flotante

0.1 + 0.2 no es exactamente 0.3. El redondeo a céntimos queda disperso por todo el código.

02

Monedas mezcladas

Nada impide sumar soles con dólares; el error aparece en la conciliación o en una fiscalización.

03

Reglas ilegibles

Un contador no puede auditar la regla, aunque SUNAT la cambie con cada resolución.

04

Errores tardíos

Los fallos se detectan en ejecución (o nunca), no cuando se escribe la regla.

Las bibliotecas de dinero validan la moneda **en ejecución**. Kipu hace del monto con moneda un **tipo verificado en compilación**.

PROPUESTA

Kipu: montos, monedas y porcentajes como tipos nativos

```
const IGV: porcentaje = 18%;
const UMBRAL: monto<PEN> = 700.00 PEN;

regla calcularIgv(base: monto<PEN>): monto<PEN> {
    retornar base * IGV;
}

var subtotal: monto<PEN> = 250.00 PEN * 3;
var igv: monto<PEN> = calcularIgv(subtotal);
var total: monto<PEN> = subtotal + igv;
si (total > UMBRAL) {
    mostrar("Detracción:", total * 12%);
}
var usd: monto<USD> = convertir(total, USD, 0.2667);
```

Sintaxis en español

Legible para quien define la regla: si, mientras, regla, retornar.

Tipos de dominio

monto<PEN> y monto<USD> son tipos distintos; solo convertir cambia la moneda.

Céntimos exactos

El código generado guarda cada monto como entero en céntimos: sin error de redondeo.

OBJETIVOS Y ESTADO

Qué pedía el Hito 2 y dónde estamos

General: construir un compilador para Kipu con ANTLR4 y LLVM que verifique en compilación la coherencia de tipos y monedas.

OE1 · COMPLETO

Léxico y construcciones

Tipos básicos más monto<M> y porcentaje.

OE2 · COMPLETO

Gramática actualizada

Forma estándar del curso y 28 derivaciones.

OE3 · COMPLETO

Lexer y parser

ANTLR4, errores en español con línea y columna.

OE4 · COMPLETO

Semántico completo

14 errores semánticos (se pedían 6).

OE5 · EN AVANCE

LLVM IR

Expresiones, control de flujo y mostrar; reglas en el Hito 3.

Además: arquitectura del compilador y plan de validación con 128 casos automáticos.

CONCEPTOS CLAVE (1/2)

Del texto al árbol: definiciones del front end

Compilador

Traduce un programa fuente a otro equivalente (objeto o intermedio) y reporta errores.

`Kipu: .kp → LLVM IR`

Token, lexema y patrón

Token: categoría (nombre + atributo). Lexema: texto que coincide. Patrón: expresión regular que lo describe.

`ID ← subtotal`

Gramática libre de contexto

G = (N, T, P, S): no terminales, terminales, producciones A → α y símbolo inicial.

`37 N, 52 T, 92 P`

Producción ε y listas

ε es la cadena vacía. Opcionales y listas se escriben con recursión y alternativa vacía.

`lista → s lista | ε`

Derivación más a la izquierda

En cada paso se reemplaza el no terminal situado más a la izquierda.

`28 derivaciones`

Árbol sintáctico y ambigüedad

El árbol representa una derivación. Una gramática es ambigua si una cadena tiene dos árboles.

`0 ambigüedades`

CONCEPTOS CLAVE (2/2)

Del árbol al código: definiciones semánticas y del back end

AST

Árbol de sintaxis abstracta: conserva solo la estructura con significado, sin símbolos auxiliares.

`Binaria(op, izq, der)`

Análisis semántico

Verifica lo que la gramática no puede expresar: declarar antes de usar, tipos compatibles.

`SEM-01 a SEM-14`

Tabla de símbolos y ámbito

Guarda nombre, categoría y tipo de cada identificador. El ámbito es la región donde es visible.

`global, regla, bloque, para`

Sistema de tipos y coerción

Reglas que dan un tipo a cada expresión; la coerción convierte de forma implícita y segura.

`entero → decimal`

LLVM IR y SSA

Representación intermedia tipada; en SSA cada valor se asigna una sola vez.

`%r = add i64 %a, %b`

Bloque básico y JIT

Bloque: instrucciones sin saltos internos. JIT: compila el IR a código máquina al ejecutar.

`--ejecutar (MCJIT)`

Siete construcciones con cinco ejemplos cada una

Construcción	Ejemplo representativo
G1 Declaración	`var subtotal: monto<PEN> = 1250.50 PEN;`
G2 Asignación (=, +=, -=)	`saldo -= 150.00 PEN;`
G3 Expresiones	`total >= 700.00 PEN y no exonerado`
G4 Selectivas	`si (esExportacion) { igv = 0.00 PEN; } sino { igv = base * 18%; }`
G5 Iterativas	`para mes desde 1 hasta 12 { total += cuota; }`
G6 Reglas, retorno y llamadas	`regla esMayor(a: entero, b: entero): booleano { retornar a > b; }`
G7 Entrada y salida	`mostrar("IGV", calcularIgv(base), "Neto", neto);`

Precedencia (mayor a menor): ( ), llamadas y convertir · menos unario · * / mod · + - · relacionales · igualdad · no · y · o

52 tokens con nombre y fragments recursivos

```
// Palabras reservadas antes que ID
SI     : 'si' ;
SINO   : 'sino' ;
MULT   : '*' ;       PUNTO_COMA : ';' ;

// Sin * ni + : recursión con alternativa vacía
ID     : LETRA ID_COLA ;
fragment ID_COLA : LETRA ID_COLA
                 | DIGITO ID_COLA
                 |
                 ;
LIT_PORCENTAJE : DIGITOS '%'
               | DIGITOS '.' DIGITOS '%' ;
```

Token: lista de lexemas

26 palabras reservadas, MONEDA (PEN, USD, EUR), 13 operadores, 7 delimitadores, 4 literales e ID.

Coincidencia más larga

<= es un solo token; sinoCaso es ID y no SINO + ID.

Cambios del Hito 2

Fragments recursivos en lugar de * y +. Se eliminó el comentario de bloque: solo existe //.

GRAMÁTICA ACTUALIZADA

Gramática reescrita en la forma estándar del curso

HITO 1 (EBNF)

```
programa : sentencia* EOF ;

declaracion
  : VAR ID ':' tipo
    ('=' expresion)? ';' ;

expresion
  : expresion op=(MULT|DIV) expresion
  | ... ;
```

HITO 2 (FORMA ESTÁNDAR)

```
programa : listaSentencias EOF ;
listaSentencias : sentencia listaSentencias
                |  ;
declaracion
  : VAR ID DOS_PUNTOS tipo inicializacion PUNTO_COMA
;
inicializacion : ASIG expresion |  ;
termino : termino MULT unario
        | termino DIV unario
        | unario ;
```

`scripts/verificar_gramatica.py`: sin literales en el parser, sin ?, *, + ni subreglas, reservadas antes que ID y sin /* */. Resultado: cumple los 6 lineamientos.

VALIDACIÓN DE LA GRAMÁTICA

28 derivaciones más a la izquierda desde el árbol de ANTLR

Ejemplo
5 por construcción

Parser ANTLR
gramática Kipu.g4

Sin ambigüedad
detección exacta de ANTLR

Preorden
derivación ⇒lm

Verificación
cada paso es una producción

```
cantidad = 5;
```

asignacion
⇒ **ID** *opAsignacion expresion* **PUNTO_COMA**
⇒ **ID ASIG** *expresion* **PUNTO_COMA**
⇒ **ID ASIG** *conjuncion* **PUNTO_COMA**
⇒ **ID ASIG** *negacion* **PUNTO_COMA**
⇒ **ID ASIG** *igualdad* **PUNTO_COMA**
⇒ **ID ASIG** *relacional* **PUNTO_COMA**
⇒ **ID ASIG** *aritmetica* **PUNTO_COMA**
⇒ **ID ASIG** *termino* **PUNTO_COMA**
⇒ **ID ASIG** *unario* **PUNTO_COMA**
⇒ **ID ASIG** *primario* **PUNTO_COMA**
⇒ **ID ASIG** *literal* **PUNTO_COMA**
⇒ **ID ASIG NUM_ENTERO PUNTO_COMA**

35 / 35 ejemplos aceptados

28 derivaciones (Anexo A)

0 ambigüedades

ANALIZADOR SEMÁNTICO COMPLETO

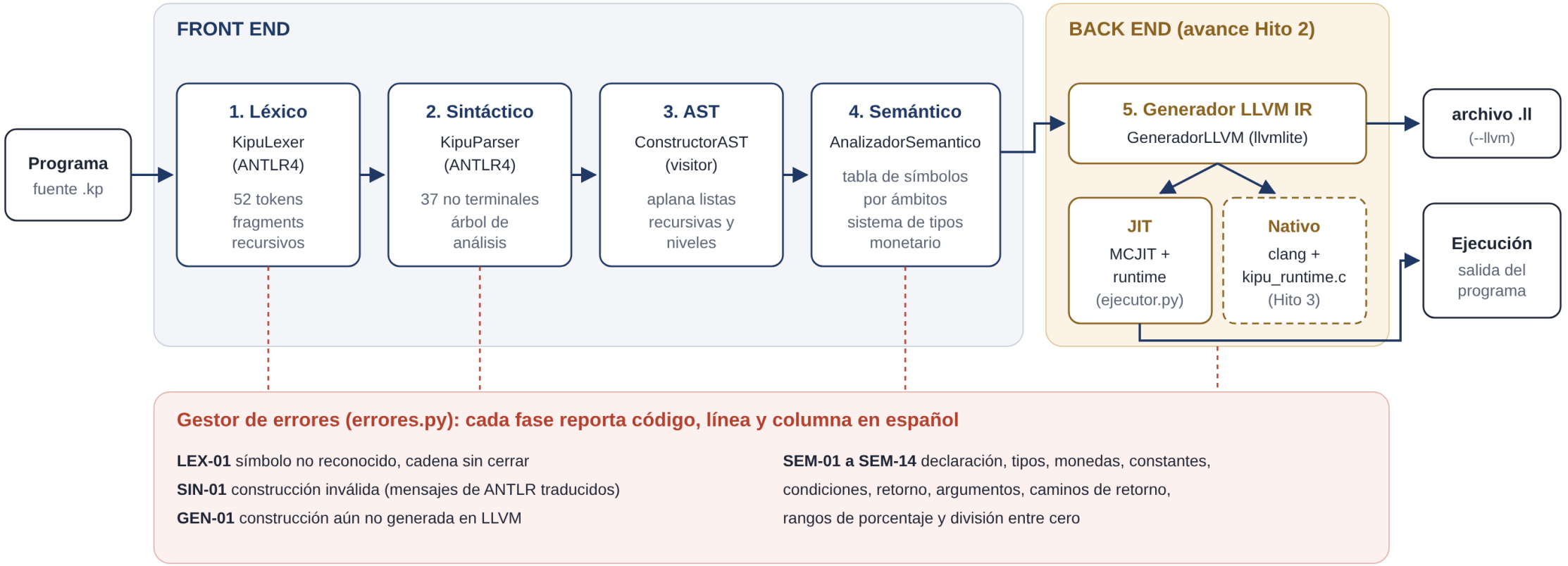
14 errores semánticos verificados (se pedían 6)

Código	Error
SEM-01	Identificador no declarado
SEM-02	Identificador redeclarado en el mismo ámbito
SEM-03	Monedas incompatibles
SEM-04	Tipos incompatibles
SEM-05	Modificación de una constante
SEM-06	Condición no booleana
SEM-07	retornar fuera de una regla

Código	Error
SEM-08	Valor de retorno incompatible
SEM-09	Número de argumentos incorrecto
SEM-10	Tipo de argumento incompatible
SEM-11	Regla que no retorna en todos los caminos
SEM-12	Porcentaje fuera de rango
SEM-13	División entre cero
SEM-14	Regla sin valor usada en una expresión

Nuevos en el Hito 2: SEM-06 a SEM-14. Cada código tiene al menos 2 casos de prueba negativos (45 errores esperados en 15 archivos). SEM-11 usa un análisis de caminos: un si sin sino no garantiza el retorno.

Una cadena de fases con un gestor de errores común



Montos como enteros de 64 bits en céntimos

Kipu	LLVM	Representación
`monto<M>`	`i64`	céntimos: 249.90 PEN → 24990
`porcentaje`	`i64`	fracción × 10^4: 18% → 1800
`entero`	`i64`	entero con signo
`decimal`	`double`	punto flotante
`booleano`	`i1`	verdadero / falso
`texto`	`i8*`	cadena constante

Generado: declaraciones, asignaciones, todas las expresiones, convertir, si / sino si / sino, mientras, para y mostrar. Pendiente para el Hito 3: reglas, llamadas y leer.

```
var igv: monto<PEN> = precio * 18%;

%prod = mul i64 %precio, 1800
%r = call i64 @kipu.div_redondeo(
         i64 %prod, i64 10000)
store i64 %r, i64* %igv

; 249.90 x 18% = 44.982 -> 44.98
; redondeo half-up exacto en enteros
```

Del programa .kp a la ejecución con el JIT de LLVM

```
$ python src/kipu.py ejemplos/detracciones.kp --ejecutar
Kipu 0.2.0 (Hito 2)  |  ejemplos/detracciones.kp
  [1/4] Análisis léxico      OK (178 tokens)
  [2/4] Análisis sintáctico  OK
  [3/4] Análisis semántico   OK
  [4/4] Generación LLVM IR   OK
--- Ejecución (JIT LLVM) ---
Factura 1 | total: 294.88 PEN | detracción: 0.00 PEN | neto: 294.88 PEN
Factura 2 | total: 589.76 PEN | detracción: 0.00 PEN | neto: 589.76 PEN
Factura 3 | total: 884.65 PEN | detracción: 106.16 PEN | neto: 778.49 PEN
Factura 4 | total: 1179.53 PEN | detracción: 141.54 PEN | neto: 1037.99 PEN
Total cobrado: 2701.12 PEN = 720.30 USD
--- Fin (código de salida 0) ---
```

Verificación manual: 3 × 249.90 = 749.70; IGV 134.946 → 134.95; total 884.65 > S/ 700, detracción 106.158 → 106.16. El ejecutable nativo (clang + runtime en C) produce la misma salida.

PLAN DE VALIDACIÓN

7 niveles de validación y 128 casos aprobados

Nivel	Qué valida	Criterio	Resultado
V1 Gramática	Lineamientos L1-L6 y ambigüedad	0 incumplimientos	36 / 36
V2 Léxico	Tokens, lexemas y LEX-01	Secuencia exacta	14 / 14
V3 Sintáctico	Programas válidos e inválidos	SIN-01 en la línea	47 / 47
V4 Derivaciones	Cada paso es una producción	28 verificadas	28 / 28
V5 Semántico	SEM-01 a SEM-14	Lista código@línea	23 / 23
V6 LLVM	IR válido y salida esperada	Salida idéntica	8 / 8
V7 Regresión	GitHub Actions (Ubuntu y Windows)	Todo aprobado	en cada push

Casos límite de redondeo: 0.05 PEN × 50% = 0.03 PEN; -0.05 PEN × 50% = -0.03 PEN; 10.00 PEN / 3 = 3.33 PEN.

PRÓXIMOS PASOS

Hacia el Hito 3: back end completo

HITO 2 · HOY

Front end completo

Gramática en forma estándar, 14 errores semánticos, arquitectura, plan de validación y LLVM IR para expresiones y control de flujo.

HITO 3 · SEMANA 15

Back end completo

Reglas, llamadas, retornar y leer en LLVM IR; ejecutables nativos con clang.

HITO 3 · SEMANA 15

Resultados

Ejecución completa del plan de validación, resultados finales y conclusiones.

Kipu: si la regla no tiene sentido contable, no compila.